

AI Assisted Coding

Lab Assignment – 13.1

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
 - Prompt AI to identify duplicate logic and replace it with functions or classes.
 - Ensure the refactored code maintains the same output.
 - Add docstrings to all functions.
- Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
 - Refactored code with a reusable function and no duplication.
 - Well documented code

```
Task-1.py x
Task-1.py > ...
22 # Refactored script with functions to remove code duplication
23 def calculate_rectangle_properties(length, width):
24     """
25     Calculate the area and perimeter of a rectangle.
26
27     Parameters:
28     length (float): The length of the rectangle.
29     width (float): The width of the rectangle.
30
31     Returns:
32     tuple: A tuple containing the area and perimeter of the rectangle.
33     """
34     area = length * width
35     perimeter = 2 * (length + width)
36     return area, perimeter
37 # List of rectangles with their dimensions
38 rectangles = [(5, 10), (7, 12), (10, 15)]
39 # Calculate and print the properties of each rectangle
40 for length, width in rectangles:
41     area, perimeter = calculate_rectangle_properties(length, width)
42     print(f"Area of Rectangle: {area}")
43     print(f"Perimeter of Rectangle: {perimeter}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console + - [] ... | [C] X

Area of Rectangle: 50
Perimeter of Rectangle: 30
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51928' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI
A-C Ass-13.1\Task-1.py'
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51928' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI
A-C Ass-13.1\Task-1.py'
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51928' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI
A-C Ass-13.1\Task-1.py'
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51928' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI
A-C Ass-13.1\Task-1.py'
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51928' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI
A-C Ass-13.1\Task-1.py'
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
 - Identify repeated or related logic and extract it into reusable functions.
 - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

Legacy script with inline repeated logic

price = 250

tax = price * 0.18

```
total = price + tax
```

```
print("Total Price:", total)
```

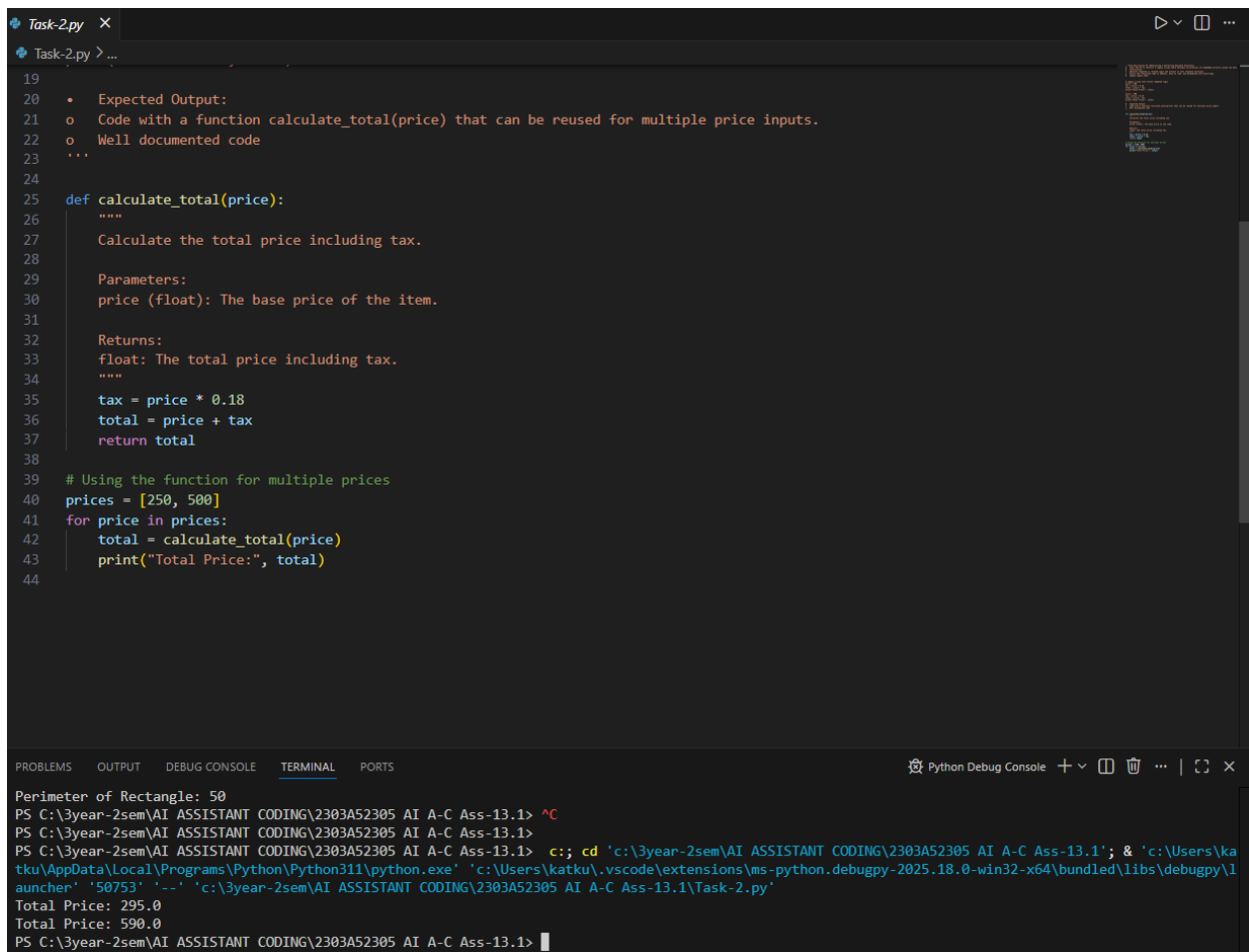
```
price = 500
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

- Expected Output:
 - Code with a function `calculate_total(price)` that can be reused for multiple price inputs.
 - Well documented code



```
Task-2.py X
Task-2.py > ...
19
20 • Expected Output:
21 ◦ Code with a function calculate_total(price) that can be reused for multiple price inputs.
22 ◦ Well documented code
23 ...
24
25 def calculate_total(price):
26     """
27     Calculate the total price including tax.
28
29     Parameters:
30     price (float): The base price of the item.
31
32     Returns:
33     float: The total price including tax.
34     """
35     tax = price * 0.18
36     total = price + tax
37     return total
38
39 # Using the function for multiple prices
40 prices = [250, 500]
41 for price in prices:
42     total = calculate_total(price)
43     print("Total Price:", total)
44
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
Perimeter of Rectangle: 50
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50753' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-2.py'
Total Price: 295.0
Total Price: 590.0
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - Accept marks as a parameter.
 - Return the corresponding grade as a string.
 - The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"
 - Marks $\geq 40 \rightarrow$ "Grade D"
 - Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
 - The class
 - The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

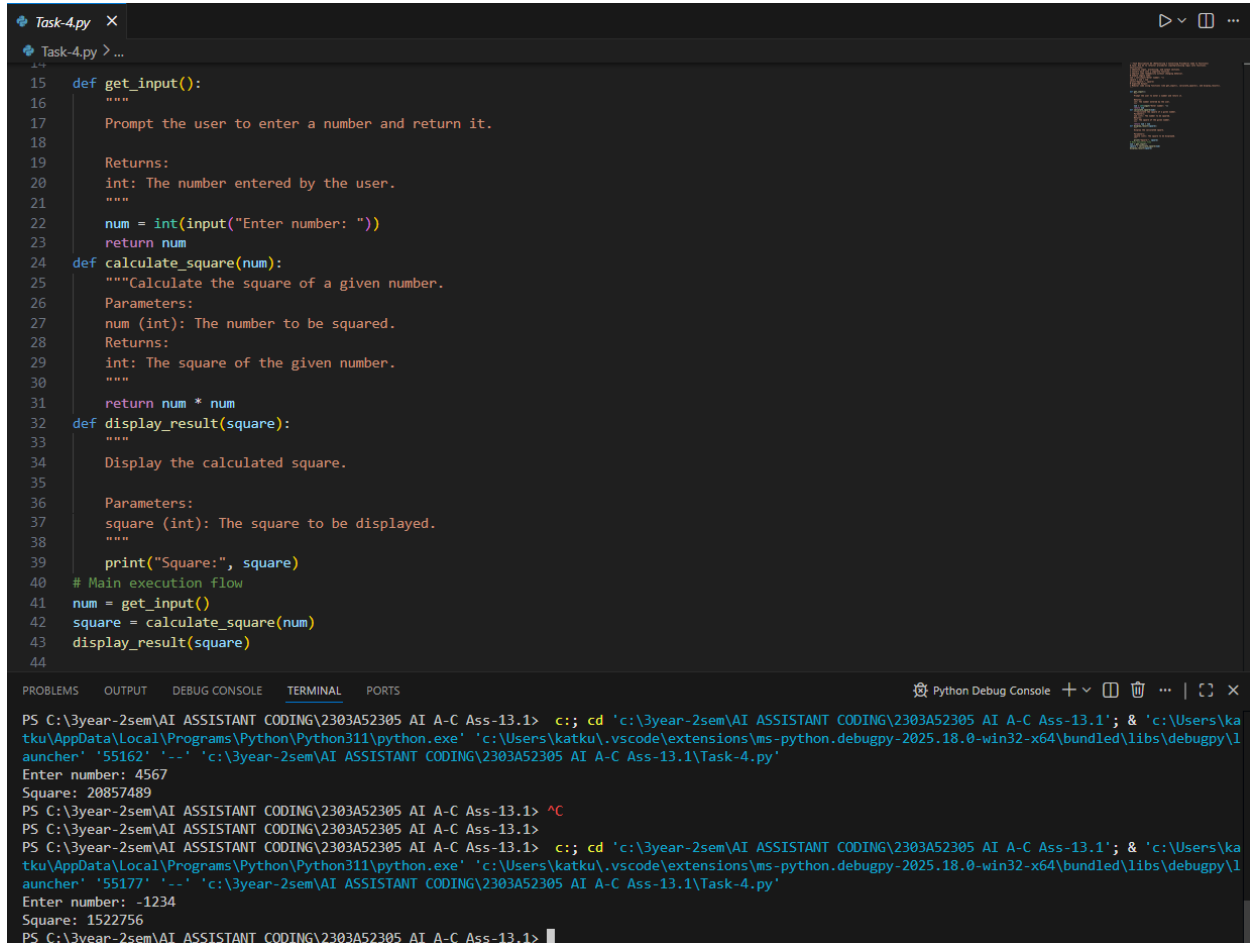
```
else:
```

```
    print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

- Expected Output:
- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.



```

15 def get_input():
16     """
17     Prompt the user to enter a number and return it.
18
19     Returns:
20     int: The number entered by the user.
21     """
22     num = int(input("Enter number: "))
23     return num
24 def calculate_square(num):
25     """Calculate the square of a given number.
26     Parameters:
27     num (int): The number to be squared.
28     Returns:
29     int: The square of the given number.
30     """
31     return num * num
32 def display_result(square):
33     """
34     Display the calculated square.
35
36     Parameters:
37     square (int): The square to be displayed.
38     """
39     print("Square:", square)
40 # Main execution flow
41 num = get_input()
42 square = calculate_square(num)
43 display_result(square)
44

```

```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '55162' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-4.py'
Enter number: 4567
Square: 20857489
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '55177' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-4.py'
Enter number: -1234
Square: 1522756
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>

```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```
Task-5.py X
Task-5.py > ...

15 class EmployeeSalaryCalculator:
16     """
17     A class to calculate the net salary of an employee after tax deduction.
18     """
19 
20     def __init__(self, salary):
21         """
22         Initialize the EmployeeSalaryCalculator with the given salary.
23 
24         Parameters:
25         salary (float): The gross salary of the employee.
26         """
27         self.salary = salary
28 
29     def calculate_net_salary(self):
30         """
31         Calculate the net salary after deducting tax.
32 
33         Returns:
34         float: The net salary after tax deduction.
35         """
36         tax = self.salary * 0.2
37         net_salary = self.salary - tax
38         return net_salary
39 
40 # Create an object of the EmployeeSalaryCalculator class
41 employee = EmployeeSalaryCalculator(int(input("Enter the gross salary: ")))
42 # Call the method to calculate net salary and print the result
43 net_salary = employee.calculate_net_salary()
44 print(net_salary)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\ka
tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '57004' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-5.py'
Enter the gross salary: 51163
40930.4
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\ka
tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '57023' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-5.py'
Enter the gross salary: 484618
387694.4
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task 6 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**
- Focus Areas:
 - o Time complexity
 - o Data structure choice

Legacy Code:


```

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

    if u == name:

        found = True

print("Access Granted" if found else "Access Denied")

```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification

The screenshot shows a VS Code editor window with a file named `Task-6.py`. The code defines a function `check_user_access` that checks if a username exists in a set of valid usernames. The main script creates a set of users, prompts the user for a username, and prints "Access Granted" or "Access Denied" based on the function's return value.

```

17
18 def check_user_access(username, user_set):
19     """
20     Check if the given username exists in the user set.
21
22     Parameters:
23     username (str): The username to check.
24     user_set (set): A set of valid usernames.
25
26     Returns:
27     bool: True if access is granted, False otherwise.
28     """
29     return username in user_set
30 # Create a set of users for O(1) average time complexity lookups
31 users = {"admin", "guest", "editor", "viewer"}
32 # Get the username input from the user
33 name = input("Enter username: ")
34 # Check access and print the result
35 if check_user_access(name, users):
36     print("Access Granted")
37 else:
38     print("Access Denied")

```

The terminal output shows the script being executed three times with different usernames: "guest", "editor", and "viewer". All three usernames are found in the set, resulting in "Access Granted" being printed each time. The fourth attempt with "sai" results in "Access Denied".

```

Enter username: guest
Access Granted
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61316' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-6.py'
Enter username: editor
Access Granted
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61333' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-6.py'
Enter username: viewer
Access Granted
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61348' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-6.py'
Enter username: sai
Access Denied

```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module `library.py` with functions:
 - `add_book(title, author, isbn)`
 - `remove_book(isbn)`
 - `search_book(isbn)`
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.

library_db = {}

# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
```

```
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```

```
Task-7.py
library_db = {}
def add_book(title, author, isbn):
    """
    Add a book to the library database.
    Parameters:
    title (str): The title of the book.
    author (str): The author of the book.
    isbn (str): The ISBN number of the book.
    Returns:
    str: A message indicating whether the book was added successfully or if it already exists.
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        return "Book added successfully."
    else:
        return "Book already exists."
def remove_book(isbn):
    """
    Remove a book from the library database.
    Parameters: isbn (str): The ISBN number of the book to be removed.
    Returns:
    str: A message indicating whether the book was removed successfully or if it was not found.
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    else:
        return "Book not found."
def search_book(isbn):
    """
    Search for a book in the library database.
    Parameters: isbn (str): The ISBN number of the book to search for.
    Returns:
    str: A message indicating whether the book was found along with its details, or if it was not found.
    """
    if isbn in library_db:
        return f"Book Found: {library_db[isbn]}"
    else:
        return "Book not found."
# Example usage of the library module
if __name__ == "__main__":
    print(add_book("Python Basics", "John Doe", "101"))
    print(add_book("AI Fundamentals", "Jane Smith", "102"))
    print(search_book("101"))
    print(remove_book("101"))
    print(search_book("101"))
```

Python Debug Console

```
auncher' '55685' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-13.1\Task-7.py'
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
PS C:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-13.1>
```

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
```

```

for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c

```

The screenshot shows a VS Code editor with a file named `Task-8.py`. The code defines a function `generate_fibonacci(n)` that generates the first `n` Fibonacci numbers. It includes docstrings for parameters and returns, and test cases that prompt the user for a limit and print the resulting series.

```

24
25 def generate_fibonacci(n):
26     """
27     Generate Fibonacci series up to n.
28
29     Parameters:
30     n (int): The number of Fibonacci numbers to generate.
31
32     Returns:
33     list: A list containing the Fibonacci series up to n.
34     """
35     if n <= 0:
36         return []
37     elif n == 1:
38         return [0]
39     elif n == 2:
40         return [0, 1]
41
42     fib_series = [0, 1]
43     for i in range(2, n):
44         c = fib_series[i - 1] + fib_series[i - 2]
45         fib_series.append(c)
46
47     return fib_series
48
49 # Test cases
50 x = int(input("Enter limit: "))
51 print(generate_fibonacci(x))

```

The terminal output shows the execution of the script. It prompts for an input limit of 101 and displays the first 101 Fibonacci numbers in a single line.

```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58690' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-8.py'
Enter limit: 101
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2178309, 3524578, 5702887, 9227465, 14930352, 24157817, 39088169, 63245986, 102334155, 165580141, 267914296, 433494437, 701408733, 1134903170, 1836311903, 2971215073, 4807526976, 7778742049, 12586269025, 20365011074, 32951280099, 53316291173, 86267571272, 139583862445, 225851433717, 365435296162, 591286729879, 956722026041, 1548008755920, 2504730781961, 4052739537881, 6557470319842, 10610209857723, 17167680177565, 27777890035288, 44945570212853, 72723460248141, 117669030460994, 190392490709135, 308061521170129, 498454011879264, 806515533049393, 1304969544928657, 2111485077978050, 3416454622906707, 5527939700884757, 8944394323791464, 14472334024676221, 23416728348467685, 37889062373143906, 61305790721611591, 99194853094755497, 160500643816367088, 259695496911122585, 420196140727489673, 679891637638612258, 1100087778366101931, 1779979416004714189, 2880067194370816120, 4660046610375530309, 7540113804746346429, 12200160415121876738, 19740274219868223167, 3194043463499009905, 51680708854858323072, 83621143489848422977, 135301852344706746049, 21892299583455169026, 354224848179261915075]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>

```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

twin primes bad version

a=11

b=13

fa=0

```
for i in range(2,a):
```

```
if a%i==0:
```

fa=1

fb=0

```
for i in range(2,b):
```

```
if b%i==0:
```

fb=1

```
if fa==0 and fb==0 and abs(a-b)==2:
```

```
print("Twin Primes")
```

else:

```
print("Not Twin Primes")
```

```
Task-9.py
Task-9.py > is_prime

29 def is_prime(n):
30     """
31     Check if a number is prime.
32     Parameters:
33     n (int): The number to check for primality.
34     Returns:
35     bool: True if n is prime, False otherwise.
36     """
37     if n <= 1:
38         return False
39     for i in range(2, int(n**0.5) + 1):
40         if n % i == 0:
41             return False
42     return True
43
44 def is_twin_prime(p1, p2):
45     """Check if two numbers are twin primes."""
46     if is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2:
47         return True
48     return False
49
50 def generate_twin_primes(limit):
51     """Generate a list of twin primes up to a given limit.
52     Parameters:
53     limit (int): The upper limit for generating twin primes.
54     Returns:
55     list: A list of tuples, each containing a pair of twin primes.
56     """
57     twin_primes = []
58     for num in range(2, limit):
59         if is_twin_prime(num, num + 2):
60             twin_primes.append((num, num + 2))
61     return twin_primes
62
63 # Get the limit input from the user
64 limit = int(input("Enter the upper limit for twin primes: "))
65 # Generate and print the list of twin primes
66 twin_prime_list = generate_twin_primes(limit)
67 print("Twin Primes up to", limit, ":", twin_prime_list)
68
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
tku\AppData\Local\Programs\Python\Python311\python.exe 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '58716' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-9.py'
Enter the upper limit for twin primes: 354
Twin Primes up to 354 : [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73), (101, 103), (107, 109), (137, 139), (149, 151), (1
79, 181), (191, 193), (197, 199), (227, 229), (239, 241), (269, 271), (281, 283), (311, 313), (347, 349)]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
```

```

    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")

```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

The screenshot shows a VS Code editor with a file named `Task-10.py`. The code defines a function `get_zodiac(year)` that returns the Chinese Zodiac sign for a given year. The function uses a list `zodiac_signs` to map years to signs. The `main()` function handles user input and prints the result. The terminal shows the script being executed, with input years 2026, 2025, and 2024, and corresponding outputs: Horse, Snake, and Dragon.

```

56
57 def get_zodiac(year):
58     """
59     Get the Chinese Zodiac sign for a given year.
60
61     Parameters:
62     year (int): The year for which to determine the Chinese Zodiac sign.
63
64     Returns:
65     str: The corresponding Chinese Zodiac sign.
66     """
67     zodiac_signs = [
68         "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
69         "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
70     ]
71     return zodiac_signs[year % 12]
72
73 def main():
74     """Main function to handle user input and display the Chinese Zodiac sign.
75     """
76     year = int(input("Enter a year: "))
77     zodiac_sign = get_zodiac(year)
78     print(f"The Chinese Zodiac sign for the year {year} is: {zodiac_sign}")
79
80 if __name__ == "__main__":
81     main()

```

Terminal Output:

```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54671' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2026
The Chinese Zodiac sign for the year 2026 is: Horse
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56529' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2025
The Chinese Zodiac sign for the year 2025 is: Snake
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56546' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2024
The Chinese Zodiac sign for the year 2024 is: Dragon
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>

```


Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$  (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$  (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - Accept an integer parameter.
 - Return True if the number is divisible by the sum of its digits.
 - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).


```

while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)

```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer `n`.
 - o Return the number of trailing zeros in `n!`.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

The screenshot shows a VS Code editor with a file named `Task-12.py`. The code defines a function `count_trailing_zeros(n)` that calculates the number of trailing zeros in `n!` by counting the number of multiples of 5 up to `n`. The function includes a docstring, a type hint for `n`, and a `ValueError` for non-negative integers. Below the function, there is an example usage section that prompts the user for a non-negative integer and prints the result.

```

38
39
40 def count_trailing_zeros(n):
41     """
42     Count the number of trailing zeros in n! (factorial of n).
43
44     Parameters:
45     n (int): A non-negative integer for which to count trailing zeros in its factorial.
46
47     Returns:
48     int: The number of trailing zeros in n!.
49     """
50     if n < 0:
51         raise ValueError("Input must be a non-negative integer.")
52
53     count = 0
54     power_of_5 = 5
55
56     while n >= power_of_5:
57         count += n // power_of_5
58         power_of_5 *= 5
59
60     return count
61
62 # Example usage
63 try:
64     n = int(input("Enter a non-negative integer: "))
65     print("Trailing zeros in {}: {}".format(n, count_trailing_zeros(n)))
66 except ValueError as e:
67     print(e)

```

The terminal output shows the execution of the script. It prompts the user for a non-negative integer. When the user enters 5645, the output is "Trailing zeros in 5645!: 1409". When the user enters -436546, the output is "Input must be a non-negative integer.".

```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61983' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-12.py'
Enter a non-negative integer: 5645
Trailing zeros in 5645!: 1409
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62000' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-12.py'
Enter a non-negative integer: -436546
Input must be a non-negative integer.
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>

```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
 - Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
 - Edge: $1 \rightarrow [1]$
 - Negative: -5
 - Large: 27 (well-known long sequence)
 - Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow \text{next} = n / 2$.
 - If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Task-13.py

Task-13.py > ...

```
35
36 def collatz_sequence(n):
37     """
38     Generate the Collatz sequence for a given integer n until reaching 1.
39
40     Parameters:
41     n (int): The starting integer for the Collatz sequence.
42
43     Returns:
44     list: A list containing the Collatz sequence from n to 1.
45     """
46     if n <= 0:
47         raise ValueError("Input must be a positive integer.")
48
49     sequence = []
50
51     while n != 1:
52         sequence.append(n)
53         if n % 2 == 0:
54             n = n // 2
55         else:
56             n = 3 * n + 1
57
58     sequence.append(1) # Append the final element of the sequence
59     return sequence
60 # Example usage
61 try:
62     n = int(input("Enter a positive integer: "))
63     print(collatz_sequence(n))
64 except ValueError as e:
65     print(e)
66
```

PROBLEMS

OUTPUT

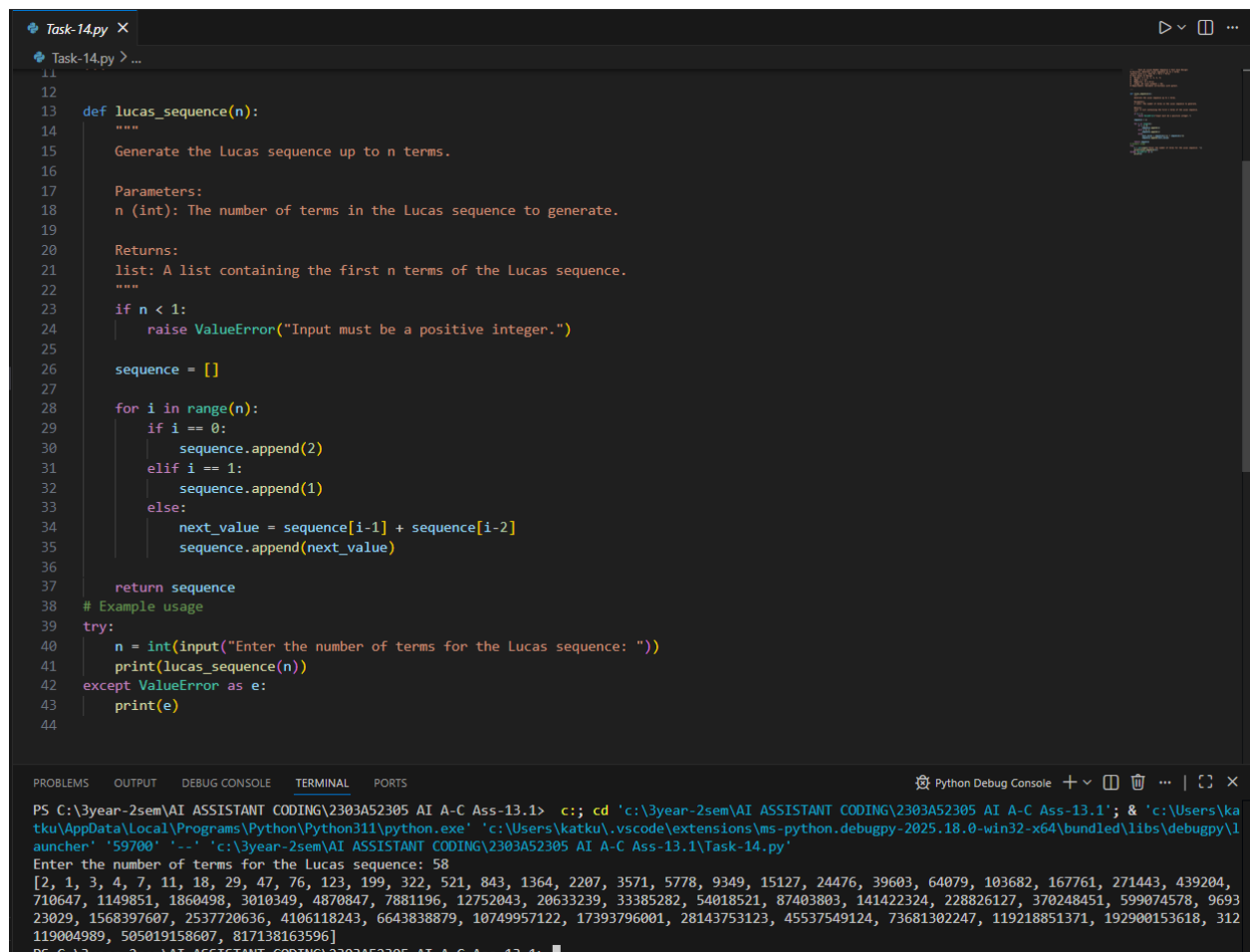
DEBUG CONSOLE

TERMINAL

PORTS

Python Debug Console

tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '59936' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-13.py'
Enter a positive integer: 866
[866, 433, 1300, 650, 325, 976, 488, 244, 122, 61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\ka
tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '59951' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-13.py'
Enter a positive integer: 2222
[2222, 1111, 3334, 1667, 5002, 2501, 7504, 3752, 1876, 938, 469, 1408, 704, 352, 176, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>



Task 15 (Vowel & Consonant Counter – Test Case Design)

- **Function:** Count vowels and consonants in string.
- **Test Cases to Design:**
 - Normal: "hello" $\rightarrow$ (2,3)
 - Edge: "" $\rightarrow$ (0,0)
 - Only vowels: "aeiou" $\rightarrow$ (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

The image shows a VS Code editor window with a Python script named `Task-15.py` and its execution output in the terminal.

Python Script (`Task-15.py`):

```
10
11 def count_vowels_consonants(s):
12     """
13     Count the number of vowels and consonants in a given string.
14
15     Parameters:
16     s (str): The input string to analyze.
17
18     Returns:
19     tuple: A tuple containing the count of vowels and consonants in the format (vowels, consonants).
20     """
21     vowels = 'aeiouAEIOU'
22     vowel_count = 0
23     consonant_count = 0
24
25     for char in s:
26         if char.isalpha(): # Check if the character is an alphabet
27             if char in vowels:
28                 vowel_count += 1
29             else:
30                 consonant_count += 1
31
32     return vowel_count, consonant_count
33
34 # Example usage
35 input_string = input("Enter a string: ")
36 vowels, consonants = count_vowels_consonants(input_string)
37 print("Vowels:", vowels)
38 print("Consonants:", consonants)
```

Terminal Output:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c::; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52897' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-15.py'
Enter a string: asdf
Vowels: 1
Consonants: 3
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> c::; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52112' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-15.py'
Enter a string: aeioiu
Vowels: 5
Consonants: 0
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
```